

TECHNICAL SKILLS

Languages: Rust, Python, C/C++, JavaScript, SQL, Java, Swift

Frameworks: Axum, Tokio, Wasmtime, FastAPI, React, Node.js, Actix-Web

Databases/Tools: Postgres, MongoDB, MySQL, Redis, Git, GitHub, Docker, Linux, Prometheus, OPA

Others: WebAssembly/WASI, seccomp BPF, Async/Await, Concurrency, Lock-Free Architecture, Ed25519, APIs, CI/CD

EDUCATION

University of Oklahoma

Master of Science in Computer Science

GPA: 4.0

Norman, OK

August 2024 – May 2026

University of Oklahoma

Bachelor of Science in Computer Science

GPA: 3.79

Norman, OK

Jan 2023 – May 2025

PROFESSIONAL EXPERIENCE

Applied Research Laboratory

Machine Learning Research Intern

Austin, TX

June 2025 – August 2025

- Eliminated the manual signal-review bottleneck in U.S. Navy Power Spectral Density (PSD) analysis by engineering a custom deep-learning architecture that automated object detection across PSD captures.
- Drove reproduction error below **0.01%** on clean PSD data by implementing advanced preprocessing pipelines and tuning model hyperparameters across iterative training runs.
- Influenced downstream research direction across senior-scientist review by delivering a final model with a comprehensive performance analysis that translated complex findings into actionable decisions.

University of Oklahoma

Teaching Assistant, Computer Organization

Norman, OK

Jan 2025 – May 2025

- Strengthened undergraduate command of memory hierarchy and I/O design by leading 1:1 and group tutoring sessions throughout the semester.
- Sustained consistent learning loops across the cohort by grading assignments, quizzes, and exams with timely, constructive feedback.

SELECTED PROJECTS

Isolated Virtual WASM Manager | *Rust, Wasmtime, Tokio, Axum, WASI, seccomp* | github.com/lucastimho/isolator-v-wasm-sandbox-bpf

- Achieved **9,685 req/s** sustained throughput at **P99 19ms** latency by engineering a 5,130-line Rust runtime that executes untrusted WASM payloads in fully isolated, ephemeral sandboxes with sub-20ms cold starts across 50 pre-warmed concurrent slots.
- Eliminated host filesystem exposure entirely by implementing a 5-layer defense-in-depth security model: seccomp BPF syscall filtering (67 allowed, 18 kill-on-invoke), OCAP SessionPolicy with 19 WASI capabilities, BTreeMap-backed virtual filesystem with per-fd capability rights and 64MB write quotas, dual-layer memory enforcement (50MB cap + RSS watchdog), and a 16-pattern compiled regex PII scrubber.
- Improved API throughput **5.4×** (1,780 → 9,685 req/s) and reduced P99 latency **13×** (249ms → 19ms) by replacing a blocking thread::sleep inside a hot-path mutex with a lock-free architecture: background tokio sampler at 500ms intervals, read-biased RwLock snapshot reads, and RAIL AdmissionTicket for real-time pool utilization tracking.
- Eliminated all guest-to-host filesystem access by intercepting 6 WASI syscalls via Wasmtime's Linker::func_wrap_async, routing I/O through an in-memory VFS with path traversal rejection and 256-fd limit, and streaming stdout to clients over SSE via 256KB ring buffers draining every 20ms.

APEX-Pay: Zero-Trust AI Agent Execution Gateway | *Python, FastAPI, PostgreSQL, OPA, Redis* | github.com/lucastimho/apex-pay
Ed25519, React, SSE

- Delivered **sub-200ms** hot-path decision latency (verified by **138 passing tests** across 10 suites) by architecting a policy-gated execution gateway in FastAPI/PostgreSQL that authorizes AI tool-calls through a 5-stage fail-closed pipeline: SHA-256 intent canonicalization, semantic risk classifier, OPA policy engine with embedded Python fallback, scoped ephemeral credential issuance, and Ed25519-signed execution receipts.
- Achieved **sub-100ms** tail latency from audit-log write to browser-rendered dashboard update by eliminating Supabase connection-pool exhaustion under concurrent SSE clients — engineered a single-connection PostgreSQL LISTEN/NOTIFY broker per uvicorn worker with asyncio.Queue fanout.
- Made the audit trail offline-verifiable and tamper-resistant by enforcing append-only semantics at two layers — three least-privilege PostgreSQL roles (apex_gateway, apex_auditor, apex_readonly) with segregated DSNs, plus a trg_audit_no_update trigger rejecting UPDATE/DELETE — and emitting Ed25519-signed receipts for every authorization decision.

Hacklahoma: Peak Performance

Best Beginners Winner

- Shipped a functional iPad AI coach in 24 hours by architecting a multi-modal pipeline for real-time video/voice metrics while mentoring 4 novice hackers.
- Sustained a responsive UI under concurrent audio and vision model inference by optimizing asynchronous workloads with Swift Actors and Async/Await.

NBA Player Archetype Clustering System

- Reached scikit-learn-competitive clustering throughput by engineering K-Means and Agglomerative algorithms from scratch in NumPy with vectorized dot-product broadcasting.
- Surfaced distinct player archetypes across 19+ statistical metrics by applying PCA and StandardScaler within a greedy trait-matching pipeline.
- Validated cluster quality against scikit-learn baselines via a Streamlit dashboard benchmarking Silhouette and Davies-Bouldin scores.

HackUTD XI Project: ToyoTrends

Second Place Winner

- Shipped a multi-vehicle carbon emission analysis tool in under 24 hours by integrating CarsXE and SambaNova AI APIs into a React.js web application.
- Surfaced actionable environmental insights by implementing dynamic Chart.js visualizations of multi-vehicle emission data.